ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		К.А. Кочин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

Работа со списками, текстурой и NURBS-поверхностями.

по курсу: КОМПЬЮТЕРНАЯ ГРАФИКА

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

1. Задание

Вывести сложный трехмерный объект, сохранить его в виде списка и размножить в виде нескольких копий различного масштаба на экране путем повторного вывода списка. Выводимые объекты располагаются над рельефом, задаваемым NURBS-поверхностью. Наложить текстуру на выводимые объекты, сцена должна быть освещена, необходимо предусмотреть возможность включения эффекта тумана (при помощи клавиатуры или мыши).

2. Текст программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <glut.h>

// Определение структуры для представления домика
struct House {
    GLfloat scale; // Масштаб
    GLfloat x, y, z; // Позиция

    House(GLfloat _scale, GLfloat _x, GLfloat _y, GLfloat _z)
        : scale(_scale), x(_x), y(_y), z(_z) {}
};

float angleX = 0.0f;
float angleY = 0.0f;
int prevMouseX, prevMouseY;
bool isRotating = false;
bool isFogEnabled = true;
GLuint textureID;
int textureWidth, textureHeight;

GLfloat ctrlpoints[4][4][3] = {
    {{-10.0, -1.0, -10.0}, {-5.0, -1.0, -10.0}, {5.0, -1.0, -10.0}, {10.0, -1.0, -10.0}},
    {{-10.0, -1.0, -5.0}, {-5.0, -1.0, -5.0}, {5.0, -1.0, -5.0}, {10.0, -1.0, -5.0}},
    {{-10.0, -1.0, 5.0}, {-5.0, -1.0, 5.0}, {5.0, -1.0, 5.0}, {10.0, -1.0, 5.0}},
    {{-10.0, -1.0, 10.0}, {-5.0, -1.0, 10.0}, {5.0, -1.0, 10.0}, {10.0, -1.0, 10.0}}
};
```

```
// Загрузка текстуры
void loadTexture() {
    glGenTextures(1, &textureID);
    glBindTexture(GL_TEXTURE_2D, textureID);

    unsigned char* data;

    // Загрузка изображения
    FILE* file = fopen("2.jpeg", "rb");
    if (!file) {
        std::cerr << "Failed to open texture file" << std::endl;
        return;
    }

    // Определение размеров текстуры без использования сторонних библиотек
    fseek(file, 0, SEEK_END);
    long fileSize = ftell(file);
    rewind(file);

    data = new unsigned char[fileSize];
    fread(data, 1, fileSize, file);
    fclose(file);

    // Определение размеров изображения
    int headerSize = 0;
    while (data[headerSize] == 0xFF) {
        headerSize++;
    }

    // Размеры текстуры
    textureWidth = data[headerSize + 12] + data[headerSize + 13] * 256;
    textureHeight = data[headerSize + 14] + data[headerSize + 15] * 256;

    // Загрузка текстуры в OpenGL
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidth, textureHeight, 0, GL_RGB,
GL_UNSIGNED_BYTE, data);

    delete[] data;
```

```
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
}

// Остальной код остается без изменений

void drawHouse() {
    // Основание домика (квадрат)
    glBegin(GL_QUADS);
    glColor3f(0.5f, 0.35f, 0.05f);
    glVertex3f(-1.0f, -1.0f, -1.0f);
    glVertex3f(1.0f, -1.0f, -1.0f);
    glVertex3f(1.0f, -1.0f, 1.0f);
    glVertex3f(-1.0f, -1.0f, 1.0f);
    glEnd();

    // Стены домика (четыре стены)
    glBindTexture(GL_TEXTURE_2D, textureID);
    glBegin(GL_QUADS);
    glColor3f(1.0f, 1.0f, 1.0f);
    glTexCoord2f(0.0f, 0.0f);
    glVertex3f(-0.8f, -1.0f, -0.8f);

    glTexCoord2f(1.0f, 0.0f);
    glVertex3f(0.8f, -1.0f, -0.8f);

    glTexCoord2f(1.0f, 1.0f);
    glVertex3f(0.8f, 0.6f, -0.8f);

    glTexCoord2f(0.0f, 1.0f);
    glVertex3f(-0.8f, 0.6f, -0.8f);
    glEnd();

    glBindTexture(GL_TEXTURE_2D, textureID);
    glBegin(GL_QUADS);
```

```
glColor3f(1.0f, 1.0f, 1.0f);
glTexCoord2f(0.0f, 0.0f);
glVertex3f(-0.8f, -1.0f, 0.8f);

glTexCoord2f(1.0f, 0.0f);
glVertex3f(0.8f, -1.0f, 0.8f);

glTexCoord2f(1.0f, 1.0f);
glVertex3f(0.8f, 0.6f, 0.8f);

glTexCoord2f(0.0f, 1.0f);
glVertex3f(-0.8f, 0.6f, 0.8f);
glEnd();

glBindTexture(GL_TEXTURE_2D, textureID);
glBegin(GL_QUADS);
glColor3f(1.0f, 1.0f, 1.0f);
glTexCoord2f(0.0f, 0.0f);
glVertex3f(-0.8f, -1.0f, -0.8f);

glTexCoord2f(1.0f, 0.0f);
glVertex3f(-0.8f, -1.0f, 0.8f);

glTexCoord2f(1.0f, 1.0f);
glVertex3f(-0.8f, 0.6f, 0.8f);

glTexCoord2f(0.0f, 1.0f);
glVertex3f(-0.8f, 0.6f, -0.8f);
glEnd();

glBindTexture(GL_TEXTURE_2D, textureID);
glBegin(GL_QUADS);
glColor3f(1.0f, 1.0f, 1.0f);
glTexCoord2f(0.0f, 0.0f);
glVertex3f(0.8f, -1.0f, -0.8f);

glTexCoord2f(1.0f, 0.0f);
glVertex3f(0.8f, -1.0f, 0.8f);
```

```
glTexCoord2f(1.0f, 1.0f);
glVertex3f(0.8f, 0.6f, 0.8f);

glTexCoord2f(0.0f, 1.0f);
glVertex3f(0.8f, 0.6f, -0.8f);
glEnd();

glBindTexture(GL_TEXTURE_2D, 0);

// Крыша (пирамида)
glBegin(GL_TRIANGLES);
glColor3f(0.8f, 0.2f, 0.2f); // Красный цвет
// Боковые грани пирамиды
glVertex3f(-0.8f, 0.6f, -0.8f);
glVertex3f(0.8f, 0.6f, -0.8f);
glVertex3f(0.0f, 1.6f, 0.0f);

glVertex3f(-0.8f, 0.6f, 0.8f);
glVertex3f(0.8f, 0.6f, 0.8f);
glVertex3f(0.0f, 1.6f, 0.0f);

// Добавляем еще две боковые грани
glVertex3f(-0.8f, 0.6f, -0.8f);
glVertex3f(0.0f, 1.6f, 0.0f);
glVertex3f(-0.8f, 0.6f, 0.8f);

glVertex3f(0.8f, 0.6f, -0.8f);
glVertex3f(0.0f, 1.6f, 0.0f);
glVertex3f(0.8f, 0.6f, 0.8f);
glEnd();

glColor3f(0.0f, 1.0f, 0.0f);

glPushMatrix();
glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4, 0, 1, 12, 4, &ctrlpoints[0][0][0]);
glEnable(GL_MAP2_VERTEX_3);
glMapGrid2f(100, 0.0, 1.0, 100, 0.0, 1.0);
```

```
    for (int j = 0; j < 100; j++) {
        glBegin(GL_LINE_STRIP);
        for (int i = 0; i <= 100; i++)
            glEvalCoord2f((GLfloat)i / 100.0, (GLfloat)j / 100.0);
        glEnd();

        glBegin(GL_LINE_STRIP);
        for (int i = 0; i <= 100; i++)
            glEvalCoord2f((GLfloat)j / 100.0, (GLfloat)i / 100.0);
        glEnd();
    }

    glPopMatrix();
}

void init() {
    glEnable(GL_LIGHTING);

    // Устанавливаем глобальные параметры фонового освещения
    //GLfloat ambientLight[] = { 0.2f, 0.2f, 0.2f, 1.0f };
    //glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight);
    // Устанавливаем позиции и параметры световых источников
    GLfloat lightIntensity1[] = { 1.5f, 0.0f, 0.0f, 1.0f }; // Красный свет
    GLfloat lightPosition1[] = { -6.0f, 0.0f, 0.0f, 1.0f }; // Красный свет
    glLightfv(GL_LIGHT1, GL_DIFFUSE, lightIntensity1);
    glLightfv(GL_LIGHT1, GL_POSITION, lightPosition1);

    GLfloat lightIntensity2[] = { 0.0f, 1.5f, 0.0f, 1.0f }; // Зеленый свет
    GLfloat lightPosition2[] = { 0.0f, 0.0f, 0.0f, 1.0f }; // Зеленый свет
    glLightfv(GL_LIGHT2, GL_DIFFUSE, lightIntensity2);
    glLightfv(GL_LIGHT2, GL_POSITION, lightPosition2);

    GLfloat lightIntensity3[] = { 0.0f, 0.0f, 1.5f, 1.0f }; // Синий свет
    GLfloat lightPosition3[] = { 6.0f, 0.0f, 2.0f, 1.0f }; // Синий свет
    glLightfv(GL_LIGHT3, GL_DIFFUSE, lightIntensity3);
    glLightfv(GL_LIGHT3, GL_POSITION, lightPosition3);
}
```

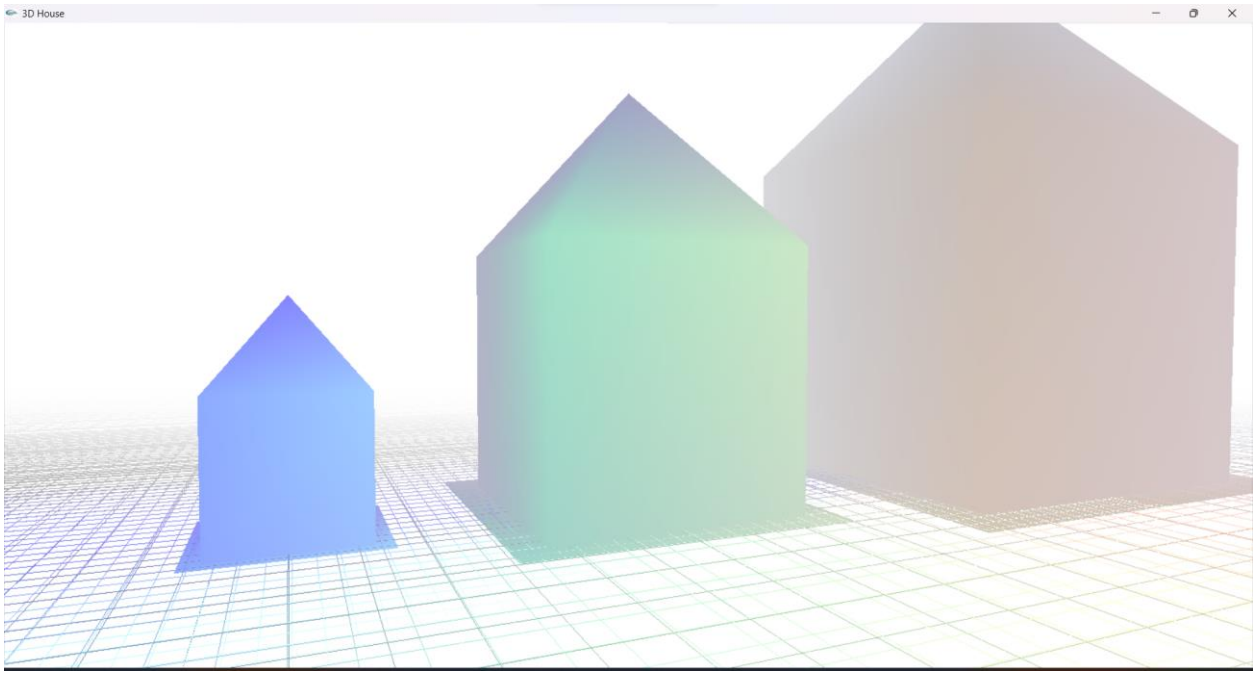
```
// Функция для рисования домика с заданными параметрами
void drawHouseWithParameters(GLfloat scale, GLfloat x, GLfloat y, GLfloat z) {
    glPushMatrix();
    glTranslatef(x, y, z);
    glScalef(scale, scale, scale);
    drawHouse();
    glPopMatrix();
}

void dim() {
    float g_FogDensity = 0.2f;
    GLfloat fogColor[4] = { 1.0f, 1.0f, 1.0f, 1.0f };
    glClearColor(1, 1, 1, 1);

    if (isFogEnabled) {
        glEnable(GL_FOG);
        glFogi(GL_FOG_MODE, GL_EXP2);
        glFogf(GL_FOG_DENSITY, g_FogDensity);
    }
    else {
        glDisable(GL_FOG);
    }

    glFogfv(GL_FOG_COLOR, fogColor);
    glHint(GL_FOG_HINT, GL_DONT_CARE);
    glFogf(GL_FOG_START, 0);
    glFogf(GL_FOG_END, 10.0f);
}

void keyboard(unsigned char key, int x, int y) {
    switch (key) {
    case 'f':
    case 'F':
        isFogEnabled = !isFogEnabled;
        dim();
        glutPostRedisplay();
        break;
```

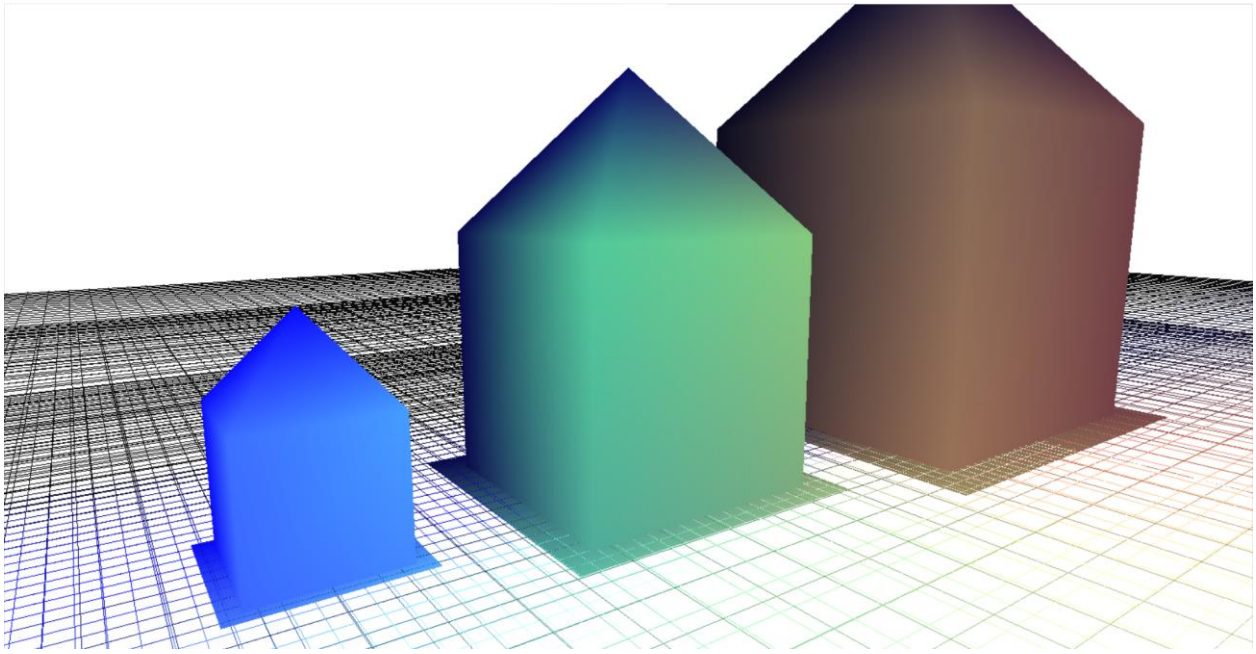
```
    case 27: // Escape key
        exit(0);
        break;
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glTranslatef(0.0f, 0.0f, -5.0f);
    glRotatef(angleX, 1.0f, 0.0f, 0.0f);
    glRotatef(angleY, 0.0f, 1.0f, 0.0f);

    // Создаем и рисуем три домика с разными параметрами
    House standardHouse(1.0f, 0.0f, 0.0f, 0.0f);
    glEnable(GL_LIGHT3);
    GLfloat lightIntensity1[] = { 1.5f, 0.0f, 0.0f, 1.0f }; // Красный свет
    GLfloat lightPosition1[] = { -2.5f, 0.0f, 0.0f, 1.0f }; // Красный свет
    glLightfv(GL_LIGHT1, GL_DIFFUSE, lightIntensity1);
    glLightfv(GL_LIGHT1, GL_POSITION, lightPosition1);
    House largerHouse(1.5f, -3.0f, 0.5f, 0.0f);
    glEnable(GL_LIGHT1);
    GLfloat lightIntensity2[] = { 0.0f, 1.5f, 0.0f, 1.0f }; // Зеленый свет
    GLfloat lightPosition2[] = { 0.0f, 0.0f, 0.0f, 1.0f }; // Зеленый свет
    glLightfv(GL_LIGHT2, GL_DIFFUSE, lightIntensity2);
    glLightfv(GL_LIGHT2, GL_POSITION, lightPosition2);
    House smallerHouse(0.5f, 2.0f, -0.5f, 0.0f);
    glEnable(GL_LIGHT2);
    GLfloat lightIntensity3[] = { 0.0f, 0.0f, 1.5f, 1.0f }; // Синий свет
    GLfloat lightPosition3[] = { 6.0f, 0.0f, 2.0f, 1.0f }; // Синий свет
    glLightfv(GL_LIGHT3, GL_DIFFUSE, lightIntensity3);
    glLightfv(GL_LIGHT3, GL_POSITION, lightPosition3);

    drawHouseWithParameters(standardHouse.scale, standardHouse.x, standardHouse.y,
standardHouse.z);
    drawHouseWithParameters(largerHouse.scale, largerHouse.x, largerHouse.y, largerHouse.z);
    drawHouseWithParameters(smallerHouse.scale, smallerHouse.x, smallerHouse.y, smallerHouse.z);
```

```
    glFlush();
    glutSwapBuffers();
}

void reshape(int width, int height) {
    glViewport(0, 0, width, height);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0f, (float)width / (float)height, 0.1f, 100.0f);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void mouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON) {
        if (state == GLUT_DOWN) {
            isRotating = true;
            prevMouseX = x;
            prevMouseY = y;
        }
        else if (state == GLUT_UP) {
            isRotating = false;
        }
    }
}

void motion(int x, int y) {
    if (isRotating) {
        int deltaX = x - prevMouseX;
        int deltaY = y - prevMouseY;
        angleX += static_cast<float>(deltaY) * 0.5f;
        angleY += static_cast<float>(deltaX) * 0.5f;
        prevMouseX = x;
        prevMouseY = y;
        glutPostRedisplay();
    }
```

```
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("3D House");
    init(); // Вызываем функцию инициализации света
    dim();
    glutKeyboardFunc(keyboard);
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMouseFunc(mouse);
    glutMotionFunc(motion);

    glEnable(GL_DEPTH_TEST);

    loadTexture(); // Загружаем текстуру

    glutMainLoop();

    return 0;
}
```

3. Скриншоты выполнения работы





4. Вывод

Этот код на языке C++ использует библиотеку GLUT для создания 3D-графики и отображения трех домиков с различными размерами. Каждый домик представляет собой куб с крышей в форме пирамиды. Создается лист домика который повторяется с разными параметрами. Освещение реализовано с помощью трех разных источников света (красный, зеленый и синий). Можно управлять вращением домиков с помощью мыши. Код создает NURBS поверхность под домиками, создавая видимость поля на котором они стоят. Так же код включает туман.